

Design and Analysis of Algorithms

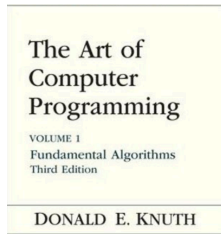
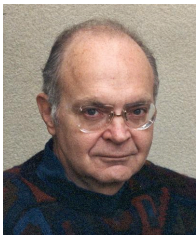
Introduction to Algorithms

- 1 A Taste of Algorithm Design
 - Return on Investment (ROI) Problem
 - Single Machine Scheduling (SMS) Problem
- 2 A Taste of Algorithm Analysis
 - Sorting Problem
- 3 A Taste of Complexity Theory
 - Travelling Salesman Problem
 - Knapsack Problem

What is An Algorithm?

“An algorithm is a finite, definite, effective procedure, with some input and some output.”

– Donald Knuth



- 1 A Taste of Algorithm Design
 - Return on Investment (ROI) Problem
 - Single Machine Scheduling (SMS) Problem

- 2 A Taste of Algorithm Analysis
 - Sorting Problem

- 3 A Taste of Complexity Theory
 - Travelling Salesman Problem
 - Knapsack Problem

Return on Investment (ROI) Problem

Problem. We have m coins to invest on n projects.

- profit function $f_i(x)$ denotes the return from investing project i with x coins, $i = 1, 2, \dots, n$.

How should we allocate the coins to maximize the total return?

Example instance: 5 coins, 4 projects:

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

Modeling

Input. $n, m, f_i(x), i \in [n], x \in \{0, \dots, m\}$

Solution. a vector $\langle x_1, x_2, \dots, x_n \rangle$, x_i is the num of coins invested on project i satisfying:

objective function: $\max \sum_{i=1}^n f_i(x_i)$

constraints: $\sum_{i=1}^n x_i = m, x_i \in \{0, \dots, m\}$

Brute-Force Algorithm: A Universal Problem-Solving Approach

Definition 1 (Brute-Force Algorithm)

A programming style that does not use any shortcuts to improve performance, but instead relies on sheer computing power to try all possibilities until the solution to a problem is found.

Brute-Force Algorithm: A Universal Problem-Solving Approach

Definition 1 (Brute-Force Algorithm)

A programming style that does not use any shortcuts to improve performance, but instead relies on sheer computing power to try all possibilities until the solution to a problem is found.

enumerate every feasible n -dimension vector $\langle x_1, x_2, \dots, x_n \rangle$ satisfying

$$x_1 + x_2 + \dots + x_n = m, x_i \in \{0, \dots, m\}$$

compute the total return

$$f_1(x_1) + f_2(x_2) + \dots + f_n(x_n)$$

find the solution with highest return

Example

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

$$x_1 + x_2 + x_3 + x_4 = 5$$

$$s_1 = \langle 0, 0, 0, 5 \rangle, v(s_1) = 24$$

$$s_2 = \langle 0, 0, 1, 4 \rangle, v(s_2) = 25$$

$$s_3 = \langle 0, 0, 2, 3 \rangle, v(s_3) = 32$$

...

$$s_{56} = \langle 5, 0, 0, 0 \rangle, v(s_{56}) = 15$$

Solution: $s = \langle 1, 0, 3, 1 \rangle$

Highest return: $11 + 30 + 20 = 61$

Efficiency of Brute-Force Algorithm

Each possible solution vector is a **non-negative integer solution** of equation

$$x_1 + x_2 + \cdots + x_n = m$$

Efficiency of Brute-Force Algorithm

Each possible solution vector is a **non-negative integer solution** of equation

$$x_1 + x_2 + \cdots + x_n = m$$

How to estimate the number of possible $\langle x_1, x_2, \dots, x_n \rangle$

- solution can be expressed as 0-1 sequence with the following format: $\# 1 = m, \# 0 = n - 1$

$$\underbrace{1 \dots 1}_{x_1} 0 \underbrace{1 \dots 1}_{x_2} 0 \dots 0 \underbrace{1 \dots 1}_{x_n}$$

$$n = 4, m = 7$$

candidate solution $\langle 1, 2, 3, 1 \rangle$ corresponds to:

$$1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1$$

Efficiency of Brute-Force Algorithm

The number of such sequences is an exponential function of input size

$$\begin{aligned}C(m + n - 1, n - 1) &= \frac{(m + n - 1)!}{m!(n - 1)!} \\ &= \Omega((1 + \epsilon)^{m+n-1})\end{aligned}$$

Efficiency of Brute-Force Algorithm

The number of such sequences is an exponential function of input size

$$\begin{aligned}C(m+n-1, n-1) &= \frac{(m+n-1)!}{m!(n-1)!} \\ &= \Omega((1+\epsilon)^{m+n-1})\end{aligned}$$

An alternative reasoning is easier to get the result: calculate the number of positive integer solutions

$$y_1 + y_2 + \cdots + y_n = m + n$$

Efficiency of Brute-Force Algorithm

The number of such sequences is an exponential function of input size

$$\begin{aligned}C(m+n-1, n-1) &= \frac{(m+n-1)!}{m!(n-1)!} \\ &= \Omega((1+\epsilon)^{m+n-1})\end{aligned}$$

An alternative reasoning is easier to get the result: calculate the number of positive integer solutions

$$y_1 + y_2 + \cdots + y_n = m + n$$

Brute-force algorithm is easy to design when the solution space is enumerable, and always correct, but not efficient when the solution space is huge.

Most of the time, we need to design “smart” algorithm.

Quiz

Q: How many trailing zeros does $1 \times 2 \times 3 \times \cdots \times 15$ have?

Quiz

Q: How many trailing zeros does $1 \times 2 \times 3 \times \cdots \times 15$ have?

- Each pair of 2 and 5 contributes one trailing 0.

Quiz

Q: How many trailing zeros does $1 \times 2 \times 3 \times \cdots \times 15$ have?

- Each pair of 2 and 5 contributes one trailing 0.
- More generally, for a prime a , the exponent of a in $N! = \prod_{i=1}^N i$ is:

$$\left\lfloor \frac{N}{a} \right\rfloor + \cdots + \left\lfloor \frac{N}{a^n} \right\rfloor, \text{ where } a^n \leq N < a^{n+1}$$

Quiz

Q: How many trailing zeros does $1 \times 2 \times 3 \times \cdots \times 15$ have?

- Each pair of 2 and 5 contributes one trailing 0.
- More generally, for a prime a , the exponent of a in $N! = \prod_{i=1}^N i$ is:

$$\left\lfloor \frac{N}{a} \right\rfloor + \cdots + \left\lfloor \frac{N}{a^n} \right\rfloor, \text{ where } a^n \leq N < a^{n+1}$$

- Since the exponent of 5 $\leq$ the exponent of 2 $\Rightarrow$ only the exponent of 5 matters, which is 3

Quiz

Q: How many numbers of the form $1a2b3c4d5e$ are divisible by 9?

Quiz

Q: How many numbers of the form $1a2b3c4d5e$ are divisible by 9?

- The count equals the number of solutions to

$$a + b + c + d + e \equiv 3 \pmod{9}$$

Quiz

Q: How many numbers of the form $1a2b3c4d5e$ are divisible by 9?

- The count equals the number of solutions to

$$a + b + c + d + e \equiv 3 \pmod{9}$$

- **Wrong:** There are 10^4 choices of (a, b, c, d) , and each choice dertermines a required value of e . However, $e \in \mathbb{Z}_{10}$, the residue class $0 \pmod{9}$ admits two valid values of e .

Quiz

Q: How many numbers of the form $1a2b3c4d5e$ are divisible by 9?

- The count equals the number of solutions to

$$a + b + c + d + e \equiv 3 \pmod{9}$$

- **Wrong:** There are 10^4 choices of (a, b, c, d) , and each choice determines a required value of e . However, $e \in \mathbb{Z}_{10}$, the residue class $0 \pmod{9}$ admits two valid values of e .
- **Correct:** As each variable ranges from 0 to 9, the sum $a + b + c + d + e$ modulo 9 forms the following sequence

$$\underbrace{0 \ 1 \ 2 \ \color{red}{3} \ 4 \ 5 \ 6 \ 7 \ 8 \mid \cdots \mid 0 \ 1 \ 2 \ \color{red}{3} \ 4 \ 5 \ 6 \ 7 \ 8 \mid 0}_{100000}$$

Hence, the count is $\lfloor 100000/9 \rfloor = 11111$.

Single Machine Scheduling Problem

Problem. n tasks, each task i requires time t_i to process (without waiting), referred to minimum processing time. We have to assign n tasks on a single machine.

- flowtime of task i : $start_i = 0$, $end_i - start_i \geq t_i$

Goal. find an assignment such that the total flowtime of all n tasks is shortest.

Single Machine Scheduling Problem

Problem. n tasks, each task i requires time t_i to process (without waiting), referred to minimum processing time. We have to assign n tasks on a single machine.

- flowtime of task i : $start_i = 0$, $end_i - start_i \geq t_i$

Goal. find an assignment such that the total flowtime of all n tasks is shortest.

数学培优新方法·四年级

——>>> 学力训练 <<<——

•• 基础 夯实 ••

1. 学校只有一个打气筒, 给一辆三轮车打足气需 7 分钟; 给一辆自行车打足气需 4 分钟; 给一辆板车打足气需 5 分钟. 同时来了三种车各一辆, 怎样安排三辆车打气的顺序, 才能使总共需要的时间(包括打气及等候的时间)最省? 最少要花多少时间?

(湖北省武汉市数学竞赛试题)

Modeling

Input.

- task set: $S = \{1, 2, \dots, n\}$
- processing time of task j : $t_j \in \mathbb{Z}^+, j \in [n]$

Output. Schedule I , a permutation of S , i.e., $(i_1, i_2, \dots, i_n)$

Objective function. the flowtime of I :

$$t(I) = \sum_{k=1}^n (n - k + 1) t_{i_k}$$

Solution. I^* — minimize $t(I^*)$

$$t(I^*) = \min\{t(I) \mid I \in \text{Permutation}(S)\}$$

Method: Greedy Algorithm

Greedy algorithm is a kind of heuristic algorithms

- construct a solution by repeatedly making a locally optimal choice
 - a greedy strategy may be motivated by intuition, but its correctness must be proved
-

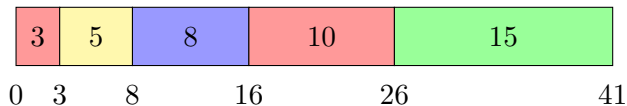
Strategy. shortest processing time (SPT) first

Algorithm. sort the processing time in an increasing order, then process them sequentially

Concrete Instance

- task set $S = \{1, 2, 3, 4, 5\}$
- minimum processing time: $t_1 = 3, t_2 = 8, t_3 = 5, t_4 = 10, t_5 = 15$

sort $(3, 8, 5, 10, 15)$ in an increasing order $\leadsto$ **Solution:** 1, 3, 2, 4, 5



overall flowtime

$$\begin{aligned}t &= 3 + (3 + 5) + (3 + 5 + 8) + (3 + 5 + 8 + 10) \\&\quad + (3 + 5 + 8 + 10 + 15) \\&= 3 \times 5 + 5 \times 4 + 8 \times 3 + 10 \times 2 + 15 \\&= 94\end{aligned}$$

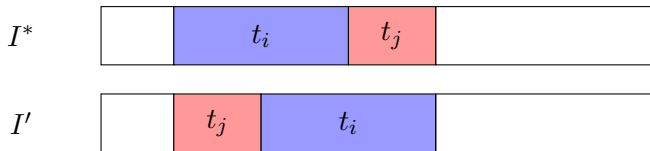
Proof of Correctness

Correctness. We have to ensure greedy algorithm yields the optimal solutions for *all instances*

Proof of Correctness

Correctness. We have to ensure greedy algorithm yields the optimal solutions for *all instances*

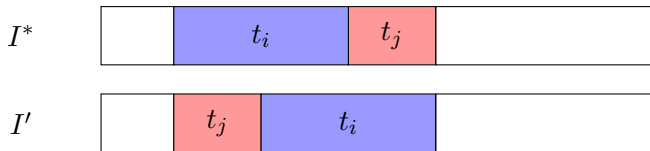
Proof. If not $\Rightarrow \exists$ optimal schedule I^* containing at least one inversion, i.e., task i and j are adjacent but $t_i > t_j$. Swap task i and j in $I^* \rightsquigarrow$ schedule I'



Proof of Correctness

Correctness. We have to ensure greedy algorithm yields the optimal solutions for *all instances*

Proof. If not $\Rightarrow \exists$ optimal schedule I^* containing at least one inversion, i.e., task i and j are adjacent but $t_i > t_j$. Swap task i and j in $I^* \rightsquigarrow$ schedule I'



flowtime comparison: $t(I') - t(I^*) = t_j - t_i < 0 \Rightarrow$ contradicting to the optimality of I^*

Heuristics Are Not Always Correct

Algorithm design cannot rely entirely on intuition. Below is a counterexample that heuristic fails.

Knapsack problem: four items need to insert into a knapsack, with values and weights as below:

label	a	b	c	d
weight w_i	3	4	5	2
value v_i	7	9	9	2

the knapsack weight limit is 6.

How to choose items to maximize the total values in the backpack?

Failure of Greedy Algorithm

Greedy strategy. highest value-to-weight ratio comes first, with weight limit 6

- sort v_i/w_i in a descending order: a, b, c, d

$$\boxed{\frac{7}{3}} > \frac{9}{4} > \frac{9}{5} > \boxed{\frac{2}{2}}$$

greedy solution: $\{a, d\}$, weight = 5, value = 9

better solution: $\{b, d\}$, weight = 6, value = 11

Summary: Steps of Algorithm Design

- 1 **Model.** Formally specify the input, output, constraints and objective function

Summary: Steps of Algorithm Design

- ① **Model.** Formally specify the input, output, constraints and objective function
- ② **Design.** Choose an appropriate design paradigm and describe the algorithm precisely

Summary: Steps of Algorithm Design

- ① **Model.** Formally specify the input, output, constraints and objective function
- ② **Design.** Choose an appropriate design paradigm and describe the algorithm precisely
- ③ **Prove correctness.** Does the algorithm produce the required output for every valid input?
 - If so, how to prove it?
 - If not, can you find a counterexample?

Summary: Steps of Algorithm Design

- ① **Model.** Formally specify the input, output, constraints and objective function
- ② **Design.** Choose an appropriate design paradigm and describe the algorithm precisely
- ③ **Prove correctness.** Does the algorithm produce the required output for every valid input?
 - If so, how to prove it?
 - If not, can you find a counterexample?
- ④ **Analyze efficiency.** Determine its computation cost (time) and storage cost (space)

Summary: Steps of Algorithm Design

- ① **Model.** Formally specify the input, output, constraints and objective function
- ② **Design.** Choose an appropriate design paradigm and describe the algorithm precisely
- ③ **Prove correctness.** Does the algorithm produce the required output for every valid input?
 - If so, how to prove it?
 - If not, can you find a counterexample?
- ④ **Analyze efficiency.** Determine its computation cost (time) and storage cost (space)

An alternative efficiency metric — the **monetary cost** to run the protocol on a cloud computing service. This new metric takes both computation cost and communication cost into consideration.

- 1 A Taste of Algorithm Design
 - Return on Investment (ROI) Problem
 - Single Machine Scheduling (SMS) Problem

- 2 A Taste of Algorithm Analysis
 - Sorting Problem

- 3 A Taste of Complexity Theory
 - Travelling Salesman Problem
 - Knapsack Problem

Insertion Algorithm

Insertion sort iterates, consuming one input element each iteration, and growing a sorted output list longer and longer.

- At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list, and inserts it there.
- It repeats until no input element remains.

input	5	7	1	3	6	2	4
intermediate state	1	3	5	6	7	2	4
after inserting 2	1	2	3	5	6	7	4

Insertion Sort: Example

input

5	7	1	3	6	2	4
---	---	---	---	---	---	---

Insertion Sort: Example

input	5	7	1	3	6	2	4
beginning	5	7	1	3	6	2	4

Insertion Sort: Example

input	5	7	1	3	6	2	4
beginning	5	7	1	3	6	2	4
insert 7	5	7	1	3	6	2	4

Insertion Sort: Example

input	5	7	1	3	6	2	4
beginning	5	7	1	3	6	2	4
insert 7	5	7	1	3	6	2	4
insert 1	1	5	7	3	6	2	4

Insertion Sort: Example

input	5	7	1	3	6	2	4
beginning	5	7	1	3	6	2	4
insert 7	5	7	1	3	6	2	4
insert 1	1	5	7	3	6	2	4
insert 3	1	3	5	7	6	2	4

Insertion Sort: Example

input	5	7	1	3	6	2	4
beginning	5	7	1	3	6	2	4
insert 7	5	7	1	3	6	2	4
insert 1	1	5	7	3	6	2	4
insert 3	1	3	5	7	6	2	4
insert 6	1	3	5	6	7	2	4

Insertion Sort: Example

input	5	7	1	3	6	2	4
beginning	5	7	1	3	6	2	4
insert 7	5	7	1	3	6	2	4
insert 1	1	5	7	3	6	2	4
insert 3	1	3	5	7	6	2	4
insert 6	1	3	5	6	7	2	4
insert 2	1	2	3	5	6	7	4

Insertion Sort: Example

input	5	7	1	3	6	2	4
beginning	5	7	1	3	6	2	4
insert 7	5	7	1	3	6	2	4
insert 1	1	5	7	3	6	2	4
insert 3	1	3	5	7	6	2	4
insert 6	1	3	5	6	7	2	4
insert 2	1	2	3	5	6	7	4
insert 4	1	2	3	4	5	6	7

Analysis of Insertion Sort

Complexity analysis

- worst-case: $O(n^2)$ comparison and swap
- best-case: $O(n)$ comparison and 0 swap
- average-case: $O(n^2)$ comparison and swap

replace array with **linked list**: eliminate swap operations (done by shift = memory access+array write) in each round

Analysis of Insertion Sort

Complexity analysis

- worst-case: $O(n^2)$ comparison and swap
- best-case: $O(n)$ comparison and 0 swap
- average-case: $O(n^2)$ comparison and swap

replace array with **linked list**: eliminate swap operations (done by shift = memory access+array write) in each round

Advantages:

- simple: Jon Bentley shows a three-line C version
- adaptive: efficient for data sets that are already substantially sorted
- stable: does not change the relative order of elements with equal keys
- in-place: requires only $O(1)$ additional memory for swap
- online: can sort a data set as the elements arrive

Analysis of Insertion Sort

Jon Bentley's three-line C version of insertion sort highlights the algorithm's simplicity, which is presented as follows:

C



```
for (i = 1; i < n; i++)  
    for (j = i; j > 0 && a[j-1] > a[j]; j--)  
        swap(a, j-1, j);
```

Analysis of Insertion Sort

Jon Bentley's three-line C version of insertion sort highlights the algorithm's simplicity, which is presented as follows:

```
C   
  
for (i = 1; i < n; i++)  
    for (j = i; j > 0 && a[j-1] > a[j]; j--)  
        swap(a, j-1, j);
```

Additional memory can be removed via the following trick

- $x := x + y; y := x - y; x := x - y.$

Analysis of Insertion Sort

Jon Bentley's three-line C version of insertion sort highlights the algorithm's simplicity, which is presented as follows:

C



```
for (i = 1; i < n; i++)  
    for (j = i; j > 0 && a[j-1] > a[j]; j--)  
        swap(a, j-1, j);
```

Additional memory can be removed via the following trick

- $x := x + y; y := x - y; x := x - y.$

Key idea: ensure each intermediate state hold shares of original values
(retains enough information for reconstructing)

Bubble Sort

Bubble sort: pass through the list — compares adjacent elements and swaps them if they are in the wrong order.

- the pass is repeated until the list is sorted
- named for the way smaller or larger elements “bubble” to the top of the list (another name is sinking sort)

before pass

5	1	6	2	8	3	4	7
---	---	---	---	---	---	---	---

one pass

1	5	2	6	3	4	7	8
---	---	---	---	---	---	---	---

Bubble Sort: Example

input

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

Bubble Sort: Example

input	5	8	1	3	6	2	4	7
pass 1	5	1	3	6	2	4	7	8

Bubble Sort: Example

input	5	8	1	3	6	2	4	7
pass 1	5	1	3	6	2	4	7	8
pass 2	1	3	5	2	4	6	7	8

Bubble Sort: Example

input	5	8	1	3	6	2	4	7
pass 1	5	1	3	6	2	4	7	8
pass 2	1	3	5	2	4	6	7	8
pass 3	1	3	2	4	5	6	7	8

Bubble Sort: Example

input	5	8	1	3	6	2	4	7
pass 1	5	1	3	6	2	4	7	8
pass 2	1	3	5	2	4	6	7	8
pass 3	1	3	2	4	5	6	7	8
pass 4	1	2	3	4	5	6	7	8

Bubble Sort: Example

input	5	8	1	3	6	2	4	7
pass 1	5	1	3	6	2	4	7	8
pass 2	1	3	5	2	4	6	7	8
pass 3	1	3	2	4	5	6	7	8
pass 4	1	2	3	4	5	6	7	8
pass 5	1	2	3	4	5	6	7	8

Analysis of Bubble Sort

Complexity analysis

- worst-case: $O(n^2)$ comparison and swap
 - best-case: $O(n)$ comparison and $O(1)$ swap
 - average-case: $O(n^2)$ comparison and swap
-

Analysis of Bubble Sort

Complexity analysis

- worst-case: $O(n^2)$ comparison and swap
 - best-case: $O(n)$ comparison and $O(1)$ swap
 - average-case: $O(n^2)$ comparison and swap
-

Advantages. simple and stable

Disadvantages. inefficient, only for education purpose

Quick Sort

QuickSort is a **divide-and-conquer** algorithm:

- ① Pick an element, called a pivot, from the array.
- ② Partitioning: reorder the array to 3 parts according to the pivot
 - low sub-array: elements smaller than the pivot
 - high sub-array: elements larger than the pivot
 - the pivot is in its final position

equal values can go either way (or stay in the middle)

- ③ Recursively apply the above steps to the sub-arrays.

The Invention of QuickSort

[FH71] Proof of a Recursive Program: Quicksort

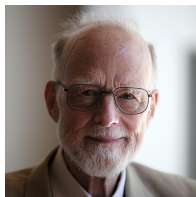


Figure: Tony Hoare

invent in 1959 while Hoare studied machine translation under Andrey Kolmogorov in Moscow State University Soviet Union

Most significant works: QuickSort and QuickSelect, Hoare logic, Communicating Sequential Processes (CSP) for concurrent processes

One Recursive Round of QuickSort

How to partition: the two-pointers technique

- left (resp. right) pointer points to element bigger (resp. smaller) than pivot
- cross happen $\leadsto$ partitioning is complete

input	5	8	1	3	6	2	4	7
-------	---	---	---	---	---	---	---	---

One Recursive Round of QuickSort

How to partition: the two-pointers technique

- left (resp. right) pointer points to element bigger (resp. smaller) than pivot
- cross happen $\leadsto$ partitioning is complete

input

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

1st swap

5	4	1	3	6	2	8	7
---	---	---	---	---	---	---	---

One Recursive Round of QuickSort

How to partition: the two-pointers technique

- left (resp. right) pointer points to element bigger (resp. smaller) than pivot
- cross happen $\leadsto$ partitioning is complete

input

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

1st swap

5	4	1	3	6	2	8	7
---	---	---	---	---	---	---	---

2nd swap

5	4	1	3	2	6	8	7
---	---	---	---	---	---	---	---

cross happens

One Recursive Round of QuickSort

How to partition: the two-pointers technique

- left (resp. right) pointer points to element bigger (resp. smaller) than pivot
- cross happen $\leadsto$ partitioning is complete

input

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

1st swap

5	4	1	3	6	2	8	7
---	---	---	---	---	---	---	---

2nd swap

5	4	1	3	2	6	8	7
---	---	---	---	---	---	---	---

cross happens

partition

2	4	1	3	5	6	8	7
---	---	---	---	---	---	---	---

One Recursive Round of QuickSort

How to partition: the two-pointers technique

- left (resp. right) pointer points to element bigger (resp. smaller) than pivot
- cross happen $\leadsto$ partitioning is complete

input

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

1st swap

5	4	1	3	6	2	8	7
---	---	---	---	---	---	---	---

2nd swap

5	4	1	3	2	6	8	7
---	---	---	---	---	---	---	---

cross happens

partition

2	4	1	3	5	6	8	7
---	---	---	---	---	---	---	---

sub
problem

4	1	3	2	5	6	8	7
---	---	---	---	---	---	---	---

Analysis of Quick Sort

Complexity analysis

Analysis of Quick Sort

Complexity analysis

- worst-case: $O(n^2)$ comparison and swap (think about when?)

Analysis of Quick Sort

Complexity analysis

- worst-case: $O(n^2)$ comparison and swap (think about when?)
totally ordered (no swap is needed) or unordered

Analysis of Quick Sort

Complexity analysis

- worst-case: $O(n^2)$ comparison and swap (think about when?)
totally ordered (no swap is needed) or unordered
- best-case: $O(n \log n)$ comparison and $O(1)$ swap
- average-case: $O(n \log n)$ comparison and swap

Analysis of Quick Sort

Complexity analysis

- worst-case: $O(n^2)$ comparison and swap (think about when?)
totally ordered (no swap is needed) or unordered
- best-case: $O(n \log n)$ comparison and $O(1)$ swap
- average-case: $O(n \log n)$ comparison and swap

Advantages

- quick: gained widespread adoption, e.g., (i) in Unix as the default library sort subroutine; (ii) it lent its name to the C standard library subroutine `qsort`; (iii) in the reference implementation of Java.

Properties

- unstable
- the choice of pivot affects performance

Merge Sort

Merge sort is also a **divide-and-conquer** algorithm:

- divide the unsorted list into n sublists, each containing one element (a list of one element is considered sorted).
- repeatedly merge sublists to produce new sorted sublists until there is only one sublist remaining. (this will be the sorted list.)

Canonical case $n = 2^k$



Figure: John von Neumann

Merge Sort: Example

input

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

Merge Sort: Example

input

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

1st merge

5	8	1	3	2	6	4	7
---	---	---	---	---	---	---	---

Merge Sort: Example

input

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

1st merge

5	8	1	3	2	6	4	7
---	---	---	---	---	---	---	---

2nd merge

1	3	5	8	2	4	6	7
---	---	---	---	---	---	---	---

Merge Sort: Example

input

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

1st merge

5	8	1	3	2	6	4	7
---	---	---	---	---	---	---	---

2nd merge

1	3	5	8	2	4	6	7
---	---	---	---	---	---	---	---

3rd merge

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

Analysis of Merge Sort

Complexity analysis

- worst-case, best-case, average-case: $O(n \log n)$ comparison
 - space: $O(n)$ total with $O(n)$ auxiliary (not in-place)
-

Advantages

- quick: (i) Linux kernel for linked list; (ii) Android platform; (iii) default sort algorithm in python and Java

Property

- stable

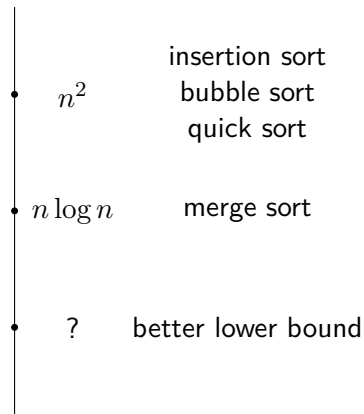
Comparisons Among Sorting Algorithms

Algorithm	worst case	best case	average case	stable
insertion sort	$O(n^2)$	$O(n)$	$O(n^2)$	yes
bubble sort	$O(n^2)$	$O(n)$	$O(n^2)$	yes
quick sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	no
merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	yes

Complexity Analysis

Which algorithm performs best? How to evaluate it?

Can we design better sorting algorithm?



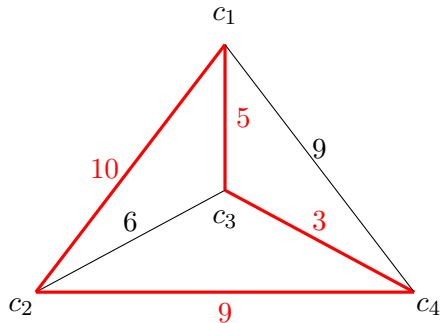
- 1 A Taste of Algorithm Design
 - Return on Investment (ROI) Problem
 - Single Machine Scheduling (SMS) Problem

- 2 A Taste of Algorithm Analysis
 - Sorting Problem

- 3 A Taste of Complexity Theory
 - Travelling Salesman Problem
 - Knapsack Problem

Travelling Salesman Problem (TSP)

Problem. Given n cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city?



Formalization

Input. Finite set of cities $C = \{c_1, c_2, \dots, c_n\}$, distance $d(c_i, c_j) = d(c_j, c_i) \in \mathbb{Z}^+$, $1 \leq i < j \leq n$.

Solution. A permutation of $1, 2, \dots, n$, a.k.a. $k_1, k_2, \dots, k_n$ such that:

$$\min \left\{ \sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1}) \right\}$$

Formalization

Input. Finite set of cities $C = \{c_1, c_2, \dots, c_n\}$, distance $d(c_i, c_j) = d(c_j, c_i) \in \mathbb{Z}^+$, $1 \leq i < j \leq n$.

Solution. A permutation of $1, 2, \dots, n$, a.k.a. $k_1, k_2, \dots, k_n$ such that:

$$\min \left\{ \sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1}) \right\}$$

Can the objective function be simpler?

- use indices modulo n — $0, 1, \dots, n-1$

$$\min \left\{ \sum_{i=0}^{n-1} d(c_{k_i}, c_{k_{i+1}}) \right\}$$

About TSP

TSP (first formulated in 1930) is one of the most extensively studied problems
 $\mathcal{NP}$ -hard problem in combinatorial optimization and theoretical computer science.

About TSP

TSP (first formulated in 1930) is one of the most extensively studied problems $\mathcal{NP}$ -hard problem in combinatorial optimization and theoretical computer science.

TSP is used as a **benchmark** for many optimization methods. Though TSP is computationally difficult, many heuristics and approximated algorithms are known.

- some instances with tens of thousands of cities can be solved completely
- even problems with millions of cities can be approximated within a small fraction of 1%.

About TSP

TSP (first formulated in 1930) is one of the most extensively studied problems $\mathcal{NP}$ -hard problem in combinatorial optimization and theoretical computer science.

TSP is used as a **benchmark** for many optimization methods. Though TSP is computationally difficult, many heuristics and approximated algorithms are known.

- some instances with tens of thousands of cities can be solved completely
- even problems with millions of cities can be approximated within a small fraction of 1%.

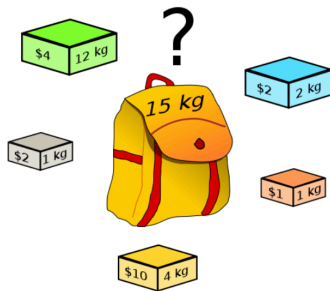
TSP has several applications

- in its purest formulation: planning, logistics, and microchip manufacturing
- slightly modified: DNA sequencing

Knapsack Problem

Given n items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit W and the total value is as large as possible.

- name: someone who is constrained by a fixed-size knapsack and must fill it with the most valuable items
- 0-1 variant: for each item, include or not



Formalization

Solution. vector $\langle x_1, x_2, \dots, x_n \rangle$ over $\{0, 1\}^n$, $x_i = 1$ iff item i is included

$$\text{objective function: } \max \sum_{i=1}^n v_i x_i$$

$$\text{constraint: } \sum_{i=1}^n w_i x_i \leq W, x_i \in \{0, 1\}, i \in [n]$$

About Knapsack Problem

Knapsack (since 1897) often arises in resource allocation where the decision makers have to choose from a set of *indivisible* projects or tasks under a fixed budget or time constraint, respectively. It is a $\mathcal{NP}$ -complete problem.

About Knapsack Problem

Knapsack (since 1897) often arises in resource allocation where the decision makers have to choose from a set of *indivisible* projects or tasks under a fixed budget or time constraint, respectively. It is a $\mathcal{NP}$ -complete problem.

Hardness of the knapsack problem depends on the input instances.

- one theme in research is to identify “hard” instances: identify what properties of instances might make them more amenable than their worst-case $\mathcal{NP}$ -complete hardness suggests
- application in public-key cryptosystems, e.g., the Merkle-Hellman knapsack cryptosystem.

About Knapsack Problem

Knapsack (since 1897) often arises in resource allocation where the decision makers have to choose from a set of *indivisible* projects or tasks under a fixed budget or time constraint, respectively. It is a $\mathcal{NP}$ -complete problem.

Hardness of the knapsack problem depends on the input instances.

- one theme in research is to identify “hard” instances: identify what properties of instances might make them more amenable than their worst-case $\mathcal{NP}$ -complete hardness suggests
- application in public-key cryptosystems, e.g., the Merkle-Hellman knapsack cryptosystem.

The basic problem is a one-dimensional (constraint) knapsack problem

- a multiple constrained problem could consider both weight and volume of knapsack

$\mathcal{NP}$ -hard Problem

$\mathcal{NP}$ (non-deterministic polynomial-time)-hardness is a class of problems that are

- informally “at least as hard as the hardest problems in $\mathcal{NP}$ ”
- an efficient algorithm for a $\mathcal{NP}$ -hard problem implies efficient algorithms for all $\mathcal{NP}$ problem

No “efficient” algorithms found yet:

- complexity of known algorithm are at least exponential function on input size
- no one can prove the “non-existence” of efficient algorithms for those problems

Thousands of $\mathcal{NP}$ -hard problems, arise throughout science, engineering, and industry.

Summary

The significance of algorithms

Criteria of evaluating algorithms

- Efficient: uses reasonable amounts of time and memory
- Correct: produces the required output for all instances

The Scope of Algorithms

- Design technique (exemplified by SMS and ROI)
 - find an algorithm $\leadsto$ model the problem and design an algorithm
 - prove the correctness $\leadsto$ establish the algorithm's correctness
- Complexity analysis (exemplified by sorting problems)
 - count the number of basic operations
- Complexity theory (TSP and Knapsack)
 - complexity classification

Reference I



M. Foley and C. A. R. Hoare.

Proof of a recursive program: Quicksort.

Comput. J., 14(4):391–395, 1971.